SHIELDCORE

MULTI-VECTOR SECURITY LAB MANUAL

Technical Operation, Vulnerability Mechanics, and Mitigation Strategies
Made by staexl

Classification	Technical Security Laboratory Training Manual
Lab Interface	Multi-Tab Cybersecurity Security Sandbox v1.1
Developer/Brand	staexl Security Solutions
Vulnerabilities Covered	10 Vectors: DDoS, SQLi, Stored XSS, CSRF, Brute Force, MitM, Ransomware
System Target Host	www.securevault.local [IP: 10.0.4.88]
Release Date	June 2026

LABORATORY OBJECTIVE: This reference manual details the technical implementation, vulnerability mechanics, and interactive execution guidelines for the ShieldCore Multi-Vector Security Lab. Students will explore hands-on threat vectors spanning from Network Infrastructures (Layer 3/4) to Web Application flaws (Layer 7) and Terminal Device compromises, verifying the mathematical and structural feedback of mitigation configurations.

1. Laboratory Layout & Interactive Architecture

The ShieldCore Security Sandbox compiles 4 distinct training tabs into a single responsive web panel. Each tab features specialized inputs, dynamic backend code viewers, visual sandboxes, and dedicated log consoles.

1.1 DDoS & Volumetric Lab (Tab 1)

Focuses on infrastructure and application layer traffic floods. Telemetry variables (CPU, socket count, latency, success rates) are evaluated at 60Hz. Packets blocked at the firewall level are colored gray on the Canvas to show scrubbing effectiveness.

1.2 Web Vulnerabilities Lab (Tab 2)

Models server-side and client-side logic flaws. Incorporates an interactive administrative directory database search (SQLi), guestbook comment board (XSS), and a split frame comparing bank transaction forms to a cross-site malicious giveaway website (CSRF).

1.3 Authentication Security Lab (Tab 3)

Simulates automated credential stuffing and dictionary attacks. Models account lockouts, CAPTCHAs, IP throttling, and multi-factor token validations.

1.4 Network & System Lab (Tab 4)

Models localized ARP Poisoning to capture plaintext passwords, alongside terminal ransomware worms that encrypt mock directories and exfiltrate files to remote command servers.

2. Analysis of the 10 Simulated Attack Vectors

2.1 HTTP Flood (L7)

Spams dynamic database queries to consume 100% server CPU resources, queuing legitimate connections.

2.2 TCP SYN Flood (L4)

Sends streams of SYN packets from spoofed IPs. Leaves connection backlog sockets (SYN_RECV) half-open, refusing new users.

2.3 UDP Flood (L3/4)

Overwhelms firewall network bandwidth (Gbps) by flooding random ports, forcing ICMP checks.

2.4 Slowloris (L7)

Holds connection threads open by sending HTTP headers extremely slowly, starving web servers without consuming CPU.

2.5 DNS Amplification (L3/4 Reflected)

Forwards queries with forged victim IP headers to open DNS resolvers, reflecting massive amplified packet payloads to the target.

2.6 SQL Injection (SQLi)

Injects database escape parameters (e.g. ' OR '1'='1) to alter query logic, bypassing logins or dumping tables.

2.7 Stored Cross-Site Scripting (XSS)

Stores malicious script payloads (e.g. <script>) inside databases, executing in the browsers of subsequent visitors to harvest session cookies.

2.8 Cross-Site Request Forgery (CSRF)

Tricks a user browser into executing unauthorized transfers on a logged-in bank portal from a malicious external site.

2.9 Credential Stuffing & Dictionary Attacks

Automates dictionary checks against login forms to compromise accounts using compromised database wordlists.

2.10 Man-in-the-Middle (MitM) & Ransomware

MitM uses ARP Poisoning to intercept LAN traffic, capturing unencrypted passwords. Ransomware executes local directory locks, renaming files to .locked and exfiltrating data.

3. Defense Mechanisms & OSI Mapping

A proper defensive configuration requires a defense-in-depth posture. The table below maps simulated defenses to their targets:

Defense Control	OSI Layer	Mitigated Attacks	Limitation
Rate Limiting	Layer 4/7	HTTP Flood, Slowloris, Brute Force	Can be bypassed by highly distributed botnets
L7 WAF Rules	Layer 7	HTTP Flood, SQLi, XSS, Slowloris	Adds server CPU overhead; bypassed by L3/4 packets
Prepared Statements	Database	SQL Injection (All types)	Does not prevent directory guessing or XSS storage
Anti-CSRF Tokens	Layer 7	Cross-Site Request Forgery	Requires active state management and cookie audits
Cloud Scrubbing	Network	UDP Flood, DNS Amplification, SYN Flood	Higher cost; adds small latency to standard packets
TLS/HTTPS Encryption	Session/7	Man-in-the-Middle sniffing	Does not prevent ransomware execution on terminals

4. Step-by-Step Training Scenarios

Scenario A: SQL Injection & Parameterization

- Navigate to **Web Vulnerabilities** tab and select **SQL Injection**.
- Observe the backend code. Click **Auth Bypass** template. Click Search. Notice that all database codes are leaked.
- Toggle **Parameterized Queries**. Look at the backend code. Run search again. Notice that zero records are returned (safe).

Scenario B: CSRF Bank Transfers & Tokens

- In **Web Vulnerabilities** tab, select **Cross-Site Request Forgery**.
- Observe your current bank balance of $10,000 on the left mock browser. Now, click the red **Claim free prize** button on the right promotional browser.
- The balance immediately drops to $7,000. Look at the console logs showing the session cookie blind authentication hijack.
- Toggle **Anti-CSRF Tokens**. Click claim prize again. The transfer fails with a 403 Forbidden code.

Scenario C: Credential Stuffing & Lockouts

- Go to **Authentication** tab. Select **Dictionary Attack**. Press **Start Attack**.
- Watch the scan monitor rapidly try leaked passwords. The database matches 'password123' and access is compromised.
- Toggle **Account Lockout**. Reset and start attack. Notice that the portal logs a lock warning and blocks scanning after 3 failures.

Scenario D: ARP Spoofing & HTTPS

- Go to **Network & System** tab. Select **Man-in-the-Middle**. Press **Engage Attack**.
- Watch the animation route packet flows through the attacker. Look at the sniffer logs dumping client credentials in plaintext.

- Toggle **HTTPS Encryption**. Packets turn green. Sniffer logs readouts show encrypted strings. Credentials remain secure.

5. Conclusion

The ShieldCore sandbox demonstrates that modern security requires target-specific controls. Applying parameterized database statements is the only standard protection against SQL injections, just as cloud scrubbing centers are required to handle volumetric UDP or reflected DNS amplification attacks. This laboratory provides developers and engineers a safe, zero-risk interface to test, visualize, and understand these critical cyber threats.